3. It replies `Done` to the manager.
4. It stops, processes no further messages, and is removed from memory.

You've seen all this before, so here is the code without further explanation.

Listing 3.3 Worker behavior

```
object Worker {

  sealed trait Command                                              ❶
  final case class Parse(replyTo: ActorRef[Worker.Response],        ❶
  ➥text: String)                                                   ❶
      extends Command                                               ❶

  sealed trait Response                                             ❶
  final case class Done(text: String) extends Response              ❶

  def apply(): Behavior[Command] =
    Behaviors.receive { (context, message) =>
      message match {
        case Parse(replyTo, task) =>
          val parsed = naiveParsing(text)                           ❷
          context.log.info(
            s"'${context.self}' DONE!. Parsed result: $parsed")     ❸
          replyTo ! Worker.Done(text)                               ❹
          Behaviors.stopped                                         ❺
      }
    }

  private def naiveParsing(text: String): String =                  ❻
    text.replaceAll("-", "")
}
```

❶ Protocol

❷ Parses the text

❸ Prints the result

❹ Replies to the manager

❺ The actor stops.